

CAP455 – OBJECT-ORIENTED PROGRAMMING USING C++

MINOR PROJECT – CA1 REPORT

AMUSEMENT PARK RIDE QUEUE SIMULATOR

Particulars	Details
Student Name	Anish Kumar
Registration / Roll Number	12626434
Programme	MCA – Core
Course Code and Name	CAP455 – Object-Oriented Programming Using C++
CA Phase	CA1
Faculty Name	Adarsh Singh
Academic Session	2026–2027
Submission Date	06-11-2026
Repositories Link	https://github.com/anishmm098/AmusementParkRideQueueSimulator.git

DECLARATION

I, **ANISH KUMAR** bearing Registration/Roll Number **12626434,** declare that the CA1 project report titled "Amusement Park Ride Queue Simulator" is my original work completed as part of the course CAP455 – Object-Oriented Programming Using C++. The project has been developed using C++ programming concepts and represents my understanding and implementation of the requirements specified for CA1.

I further declare that the work presented in this report has been completed by me and has not been submitted previously for any other academic purpose.

Student Signature: Anish Kumar **Date: 06-11-2026**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty, Adarsh Singh, for his guidance and support throughout the development of this Minor Project – CA1. His suggestions and feedback helped me understand the requirements of the project and apply the concepts of Object-Oriented Programming using C++ effectively.

I am also thankful to my institution and course faculty for providing the necessary learning environment and resources to complete this project.

Finally, I would like to thank my friends, for their encouragement and support during the completion of this work.

TABLE OF CONTENTS

1. Introduction **3**

2. Problem Statement **3**

3. Aim and Objectives **4**

4. Scope and Limitations **4**

5. Feasibility and Requirements **5**

6. System Analysis and Design **5**

7. Data Dictionary **7**

8. Algorithms and Program Flow **8**

9. C++ Concepts Used **10**

10. Source Code Screenshot **11**

11. Testing and Test Cases **15**

12. Sample Output and Evidence **16**

13. Future Enhancements **21**

14. Conclusion **22**

1. INTRODUCTION

The Amusement Park Ride Queue Simulator is a C++ based console application developed to manage basic visitor registration and ride information in an amusement park. It provides a simple menu-driven system to add visitors, display their records, and view the total number of registered visitors.

The project is developed for Minor Project – CA1 of CAP455 – Object-Oriented Programming Using C++. It applies basic C++ and Object-Oriented Programming concepts such as classes, arrays, functions, validation, static members, pointers, and friend functions.

2. PROBLEM STATEMENT

Managing visitor and ride information manually in an amusement park can be time-consuming and may lead to errors or duplicate records. A simple computerized system

is required to register visitors, store their basic ride information, and display records efficiently.

The proposed Amusement Park Ride Queue Simulator addresses this problem by providing a menu-driven C++ application for visitor registration, record display, duplicate ticket ID checking, and visitor counting.

3. AIM AND OBJECTIVES

Aim

The main aim of this project is to develop a simple C++ based system for registering visitors and managing their basic ride information in an amusement park.

Objectives

- To register visitors with their ticket ID, name, age, and ride name.

- To display the registered visitor records.
- To maintain the total number of registered visitors.
- To prevent duplicate ticket IDs.
- To validate user input and reject invalid data.
- To apply basic Object-Oriented Programming concepts in C++.

4. SCOPE AND LIMITATIONS

4.1 Features Covered in CA1

- New visitors can be registered using their ticket ID, name, age, and selected ride.
- Registered visitor records can be viewed through the Display option.
- The system checks that each ticket ID is unique.
- Visitor name and ride name cannot be left blank.
- Age is accepted only within the range of 1 to 80 years.
- Invalid numeric entries are rejected and the user is asked to enter the value again.
- The system automatically maintains the number of successfully registered visitors.
- A fixed array is used to store up to 100 Visitor objects.

4.2 Features Outside the CA1 Scope

- Permanent storage using files or databases.
- Updating or removing an existing visitor record.
- Complete queue management and automatic ride allocation.
- Managing the maximum capacity of individual rides.
- Calculating visitor waiting time.
- User login, networking, or graphical user interface.

Note: Visitor information is maintained only during the current program execution. Once the application is closed, the stored records are removed. CA1 does not use file or database storage.

5. FEASIBILITY AND REQUIREMENTS

5.1 Technical Feasibility

The project is technically feasible because it uses basic C++ features and can run on a standard computer with a C++ compiler. The application does not require an internet connection, database, or external software.

5.2 Hardware Requirements

- Standard computer or laptop
- Minimum 4 GB RAM
- Keyboard and monitor

5.3 Software Requirements

- Windows operating system
- C++ compiler such as MinGW/G++
- Visual Studio Code or any suitable C++ IDE
- Command Prompt or terminal for program execution

5.4 Functional Requirements

- The system should allow visitor registration.
- The system should validate visitor details.
- The system should reject duplicate ticket IDs.
- The system should display registered visitors.
- The system should show the total number of visitors.

6. SYSTEM ANALYSIS AND DESIGN

6.1 User and Use Cases

The CA1 application has a single console user who interacts with the system through the main menu. The user can add visitors, display registered visitors, check the total number of visitors, or exit the application.

Use Case	Input / Action	System Response

Add Visitor	Select option 1 and enter visitor details	Validates the entered data, checks the ticket ID, stores the visitor, and updates the total count.
Display Visitors	Select option 2	Displays all registered visitor records or shows a message when no records are available.
Total Visitors	Select option 3	Displays the total number of successfully registered visitors.
Exit	Select option 4	Terminates the program.

6.2 Menu Design

The application uses a simple menu-driven interface:

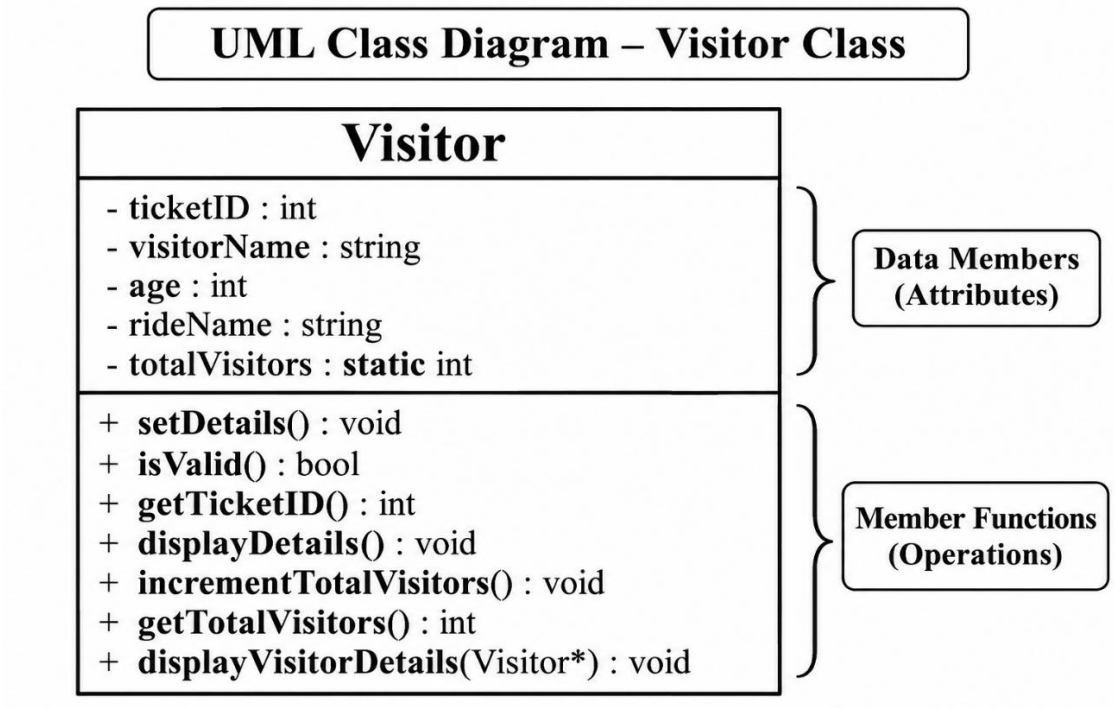
---------- AMUSEMENT PARK RIDE QUEUE SIMULATOR ----------

1. **Add Visitor**
2. **Display Visitors**
3. **Total Visitors**
4. **Exit**

Enter your choice:

6.3 UML Class Diagram

The UML class diagram represents the structure of the **Visitor class,** including its **private data members, public member functions, and friend function.**



6.4 Array and Pointer Design

```
Visitor visitors[MAX_VISITORS];
Visitor *ptr = visitors;
displayVisitorDetails(ptr + i);
```

The array can store up to 100 Visitor objects. The pointer initially points to the first object in the array. Pointer arithmetic is then used to access individual visitor objects while displaying their details.

7. DATA DICTIONARY

The data dictionary describes the main data members and functions used in the CA1 implementation.

7.1 Data Members

Name	Data Type	Purpose
ticketID	int	Stores the unique ticket ID of a visitor.
visitorName	string	Stores the visitor's name.
age	int	Stores the visitor's age.
rideName	string	Stores the selected ride name.
totalVisitors	static int	Maintains the total number of successfully registered visitors.

7.2 Important Functions

Function	Return Type	Purpose
setDetails()	void	Accepts visitor details from the user.
isValid()	bool	Checks whether the visitor data is valid.
getTicketID()	int	Returns the visitor's ticket ID.
displayDetails()	void	Displays the visitor's details.

incrementTotalVisitors()	void	Increases the total visitor count.
getTotalVisitors()	int	Returns the current visitor count.
displayVisitorDetails(Visitor *v)	void	Displays private visitor details using a pointer.

8. ALGORITHMS AND PROGRAM FLOW

The CA1 program follows a simple menu-driven flow. The user selects an operation, provides the required input, and the system performs validation before processing the request.

8.1 Algorithm for Adding a Visitor

1. Start the program and display the main menu.
2. Select **Add Visitor**.
3. Enter ticket ID, visitor name, age, and ride name.
4. Validate the entered values.
5. Check whether the ticket ID already exists.
6. If the data is invalid or the ID is duplicate, display an error message.
7. If the data is valid, store the visitor in the array.
8. Increase the total visitor count.
9. Return to the main menu.

8.2 Algorithm for Displaying Visitors

1. Select **Display Visitors** from the menu.
2. Check whether any visitor records are available.
3. If no records exist, display an appropriate message.
4. Otherwise, access each Visitor object using the array and pointer.
5. Display the details of each registered visitor.
6. Return to the main menu.

8.3 Algorithm for Total Visitors

1. Select **Total Visitors** from the main menu.
2. Retrieve the value of the static totalVisitors member.
3. Display the total number of successfully registered visitors.
4. Return to the main menu.

8.4 Program Flowchart

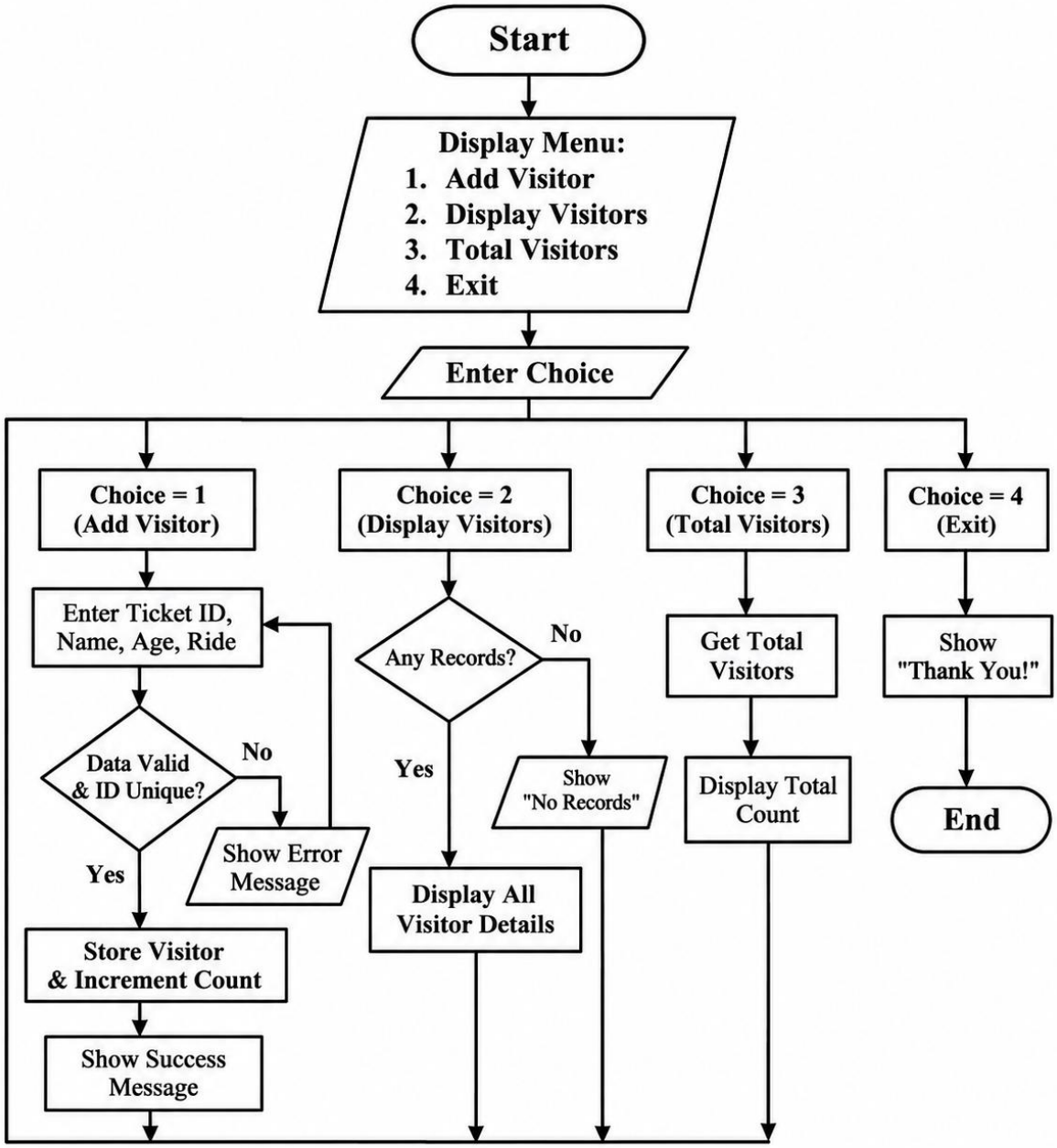


Figure 2: Program Flowchart of Amusement Park Ride Queue Simulator

9. C++ CONCEPTS USED

9.1 Class and Objects

A class named Visitor is created to store visitor information. Objects of this class are used to maintain individual visitor records.

9.2 Array of Objects

An array of 100 Visitor objects is used to store visitor records temporarily while the program is running.

9.3 Encapsulation

The data members of the Visitor class are declared private. Public member functions are used to access and manage visitor information.

9.4 Static Data Member

A static variable named totalVisitors maintains the total number of successfully registered visitors. It is shared among all objects of the Visitor class.

9.5 Inline Member Function

The displayDetails() function is defined inside the Visitor class. It demonstrates the use of an inline member function.

9.6 Friend Function

The displayVisitorDetails() function is declared as a friend of the Visitor class. It accesses private data members to display visitor information.

9.7 Pointers

A pointer of type Visitor* is used to access visitor objects stored in the array.

9.8 This Pointer

The this pointer is used inside the setDetails() function to refer to the current object.

9.9 String Handling

The string data type is used to store visitor names and ride names.

9.10 Input Validation

Input validation is implemented to reject invalid numeric values, empty names, invalid ages, and duplicate ticket IDs.

10. SOURCE CODE SCREENSHOT

The complete source code of main.cpp is shown below in screenshot form (with line numbers) for reference and verification.



```
main.cpp M
C: > Users > ANISH > OneDrive > Desktop > c++ project > AmusementParkRideQueueSimulator > src > main.cpp > Visitor
1   #include <iostream>
2   #include <string>
3   #include <sstream>
4   using namespace std;
5
6   const int MAX_VISITORS = 100;
7
8   // Function to accept valid integer input
9   int readInteger(string message)
10  {
11      string input;
12      int value;
13      char extra;
14
15      while (true)
16      {
17          cout << message;
18          getline(cin, input);
19
20          stringstream ss(input);
21
22          if (ss >> value && !(ss >> extra))
23          {
24              return value;
25          }
26
27          cout << "Invalid input! Please enter a valid number.\n";
28      }
29  }
30
31  // Visitor Class
32  class Visitor
33  {
34  private:
35      int ticketID;
36      string visitorName;
37      int age;
38      string rideName;
39
40      static int totalVisitors;
41
42  public:
43
44      // Function to enter visitor details
45      void setDetails()
46      {
47          ticketID = readInteger("Enter Ticket ID: ");
48
```

Figure 10.1 – Source code of main.cpp (lines 1–48)

```
49          cout << "Enter Visitor Name: ";
50          getline(cin, this->visitorName);
51
52          age = readInteger("Enter Age: ");
53
54          cout << "Enter Ride Name: ";
55          getline(cin, this->rideName);
56      }
57
58      // Function to validate visitor details
59      bool isValid() const
60      {
61          if (ticketID <= 0)
62              return false;
63
64          if (visitorName.find_first_not_of(" \t") == string::npos)
65              return false;
66
67          if (age <= 0 || age > 120)
68              return false;
69
70          if (rideName.find_first_not_of(" \t") == string::npos)
71              return false;
72
73          return true;
74      }
75
76      // Getter function
77      int getTicketID() const
78      {
79          return ticketID;
80      }
81
82      // Inline function to display visitor details
83      inline void displayDetails()
84      {
85          cout << "\nTicket ID: " << ticketID << endl;
86          cout << "Visitor Name: " << visitorName << endl;
87          cout << "Age: " << age << endl;
88          cout << "Ride Name: " << rideName << endl;
89      }
90
91      // Static function to increase visitor count
92      static void incrementTotalVisitors()
93      {
94          totalVisitors++;
95      }
96
97      // Static function to get total visitors
98      static int getTotalVisitors()
99      {
100         return totalVisitors;
101     }
102
103     // Friend function declaration+
104     friend void displayVisitorDetails(Visitor *v);
105 };
106
107 // Define static data member
108 int Visitor::totalVisitors = 0;
```

Figure 10.2 – Source code of main.cpp (lines 49–108)

```
109
110 // Friend function definition
111 void displayVisitorDetails(Visitor *v)
112 {
113     cout << "\n----- Visitor Details -----" << endl;
114
115     cout << "Ticket ID: " << v->ticketID << endl;
116     cout << "Visitor Name: " << v->visitorName << endl;
117     cout << "Age: " << v->age << endl;
118     cout << "Ride Name: " << v->rideName << endl;
119 }
120
121 // Function to check duplicate Ticket ID
122 bool isDuplicate(Visitor visitors[], int count, int id)
123 {
124     for (int i = 0; i < count; i++)
125     {
126         if (visitors[i].getTicketID() == id)
127         {
128             return true;
129         }
130     }
131
132     return false;
133 }
134
135 // Function to register a visitor
136 void addVisitor(Visitor visitors[], int &count)
137 {
138     if (count >= MAX_VISITORS)
139     {
140         cout << "\nVisitor storage is full!\n";
141         return;
142     }
143
144     Visitor newVisitor;
145
146     cout << "\n----- Visitor Registration -----\n";
147
148     newVisitor.setDetails();
149
150     if (!newVisitor.isValid())
151     {
152         cout << "\nInvalid visitor details! Registration failed.\n";
153         return;
154     }
155
156     if (isDuplicate(visitors, count, newVisitor.getTicketID()))
157     {
158         cout << "\nTicket ID already exists! Registration failed.\n";
159         return;
```

Figure 10.3 – Source code of main.cpp (lines 109–159)

```
160     }
161
162     visitors[count] = newVisitor;
163
164     count++;
165
166     Visitor::incrementTotalVisitors();
167
168     cout << "\nVisitor registered successfully!\n";
169 }
170
171 // Function to display all visitors
172 void displayVisitors(Visitor visitors[], int count)
173 {
174     if (count == 0)
175     {
176         cout << "\nNo visitor records available.\n";
177         return;
178     }
179
180     cout << "\n===== ALL REGISTERED VISITORS =====\n";
181
182     // Pointer to Visitor objects
183     Visitor *ptr = visitors;
184
185     for (int i = 0; i < count; i++)
186     {
187         displayVisitorDetails(ptr + i);
188     }
189
190     cout << "\nTotal Visitors: "
191          << Visitor::getTotalVisitors() << endl;
192 }
193
194 // Main function
195 int main()
196 {
197     Visitor visitors[MAX_VISITORS];
198
199     int count = 0;
200     int choice;
201
202     do
203     {
204         cout << "\n====================================\n";
205         cout << " AMUSEMENT PARK RIDE QUEUE SIMULATOR\n";
206         cout << "====================================\n";
207
208         cout << "1. Add Visitor\n";
209         cout << "2. Display Visitors\n";
210         cout << "3. Total Visitors\n";
211         cout << "4. Exit\n";
```

Figure 10.4 – Source code of main.cpp (lines 160–211)

```
211         cout << "4. Exit\n";
212
213         choice = readInteger("\nEnter your choice: ");
214
215         switch (choice)
216         {
217             case 1:
218                 addVisitor(visitors, count);
219                 break;
220
221             case 2:
222                 displayVisitors(visitors, count);
223                 break;
224
225             case 3:
226                 cout << "\nTotal Registered Visitors: "
227                      << Visitor::getTotalVisitors() << endl;
228                 break;
229
230             case 4:
231                 cout << "\nThank you for using the system!\n";
232                 break;
233
234             default:
235                 cout << "\nInvalid choice! Try again.\n";
236         }
237
238     } while (choice != 4);
239
240     return 0;
241 }
```

Figure 10.5 – Source code of main.cpp (lines 211–241)

11. TESTING AND TEST CASES

The application was tested with different valid and invalid inputs to verify visitor registration, input validation, duplicate ticket checking, record display, and visitor counting.

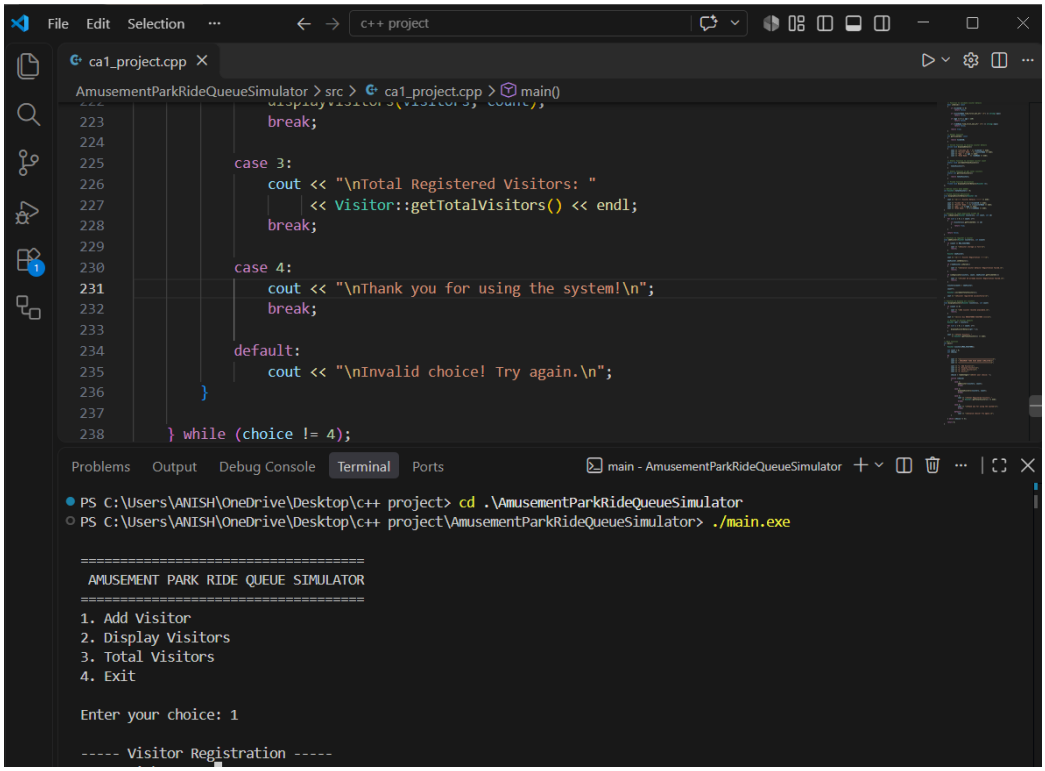
Test ID	Test Input / Action	Expected Result	Actual Result / Status
TC01	Register a visitor with valid details	Visitor should be added successfully.	Visitor added successfully – Pass
TC02	Enter an already registered ticket ID	System should reject the duplicate ID.	Duplicate ID rejected – Pass
TC03	Enter age as 150	System should reject the invalid age.	Invalid age rejected – Pass

TC04	Enter invalid text instead of a number	System should ask for valid numeric input.	User was prompted again – Pass
TC05	Select **Display Visitors** after registration	Stored visitor details should be displayed.	Visitor records displayed – Pass
TC06	Register more than one visitor	All valid records should be stored correctly.	Multiple visitors handled correctly – Pass
TC07	Select **Total Visitors**	System should display the correct visitor count.	Correct total displayed – Pass
TC08	Display visitors without registering anyone	System should show that no records are available.	Empty-record message displayed – Pass

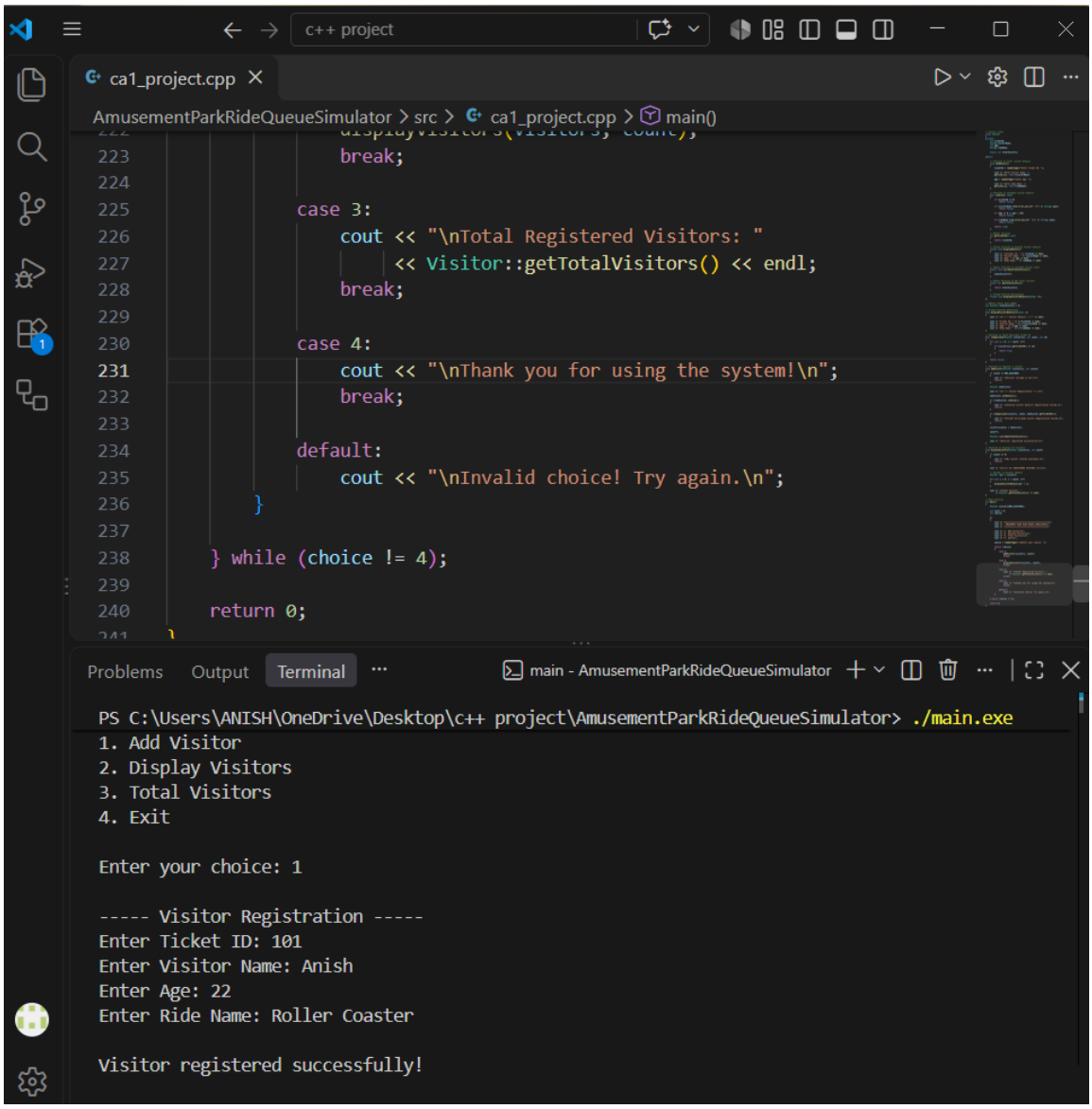
12. SAMPLE OUTPUT AND EVIDENCE

The following screenshots provide evidence of the working CA1 implementation. They show the main menu, visitor registration, record display, validation, and visitor count operations performed during testing.

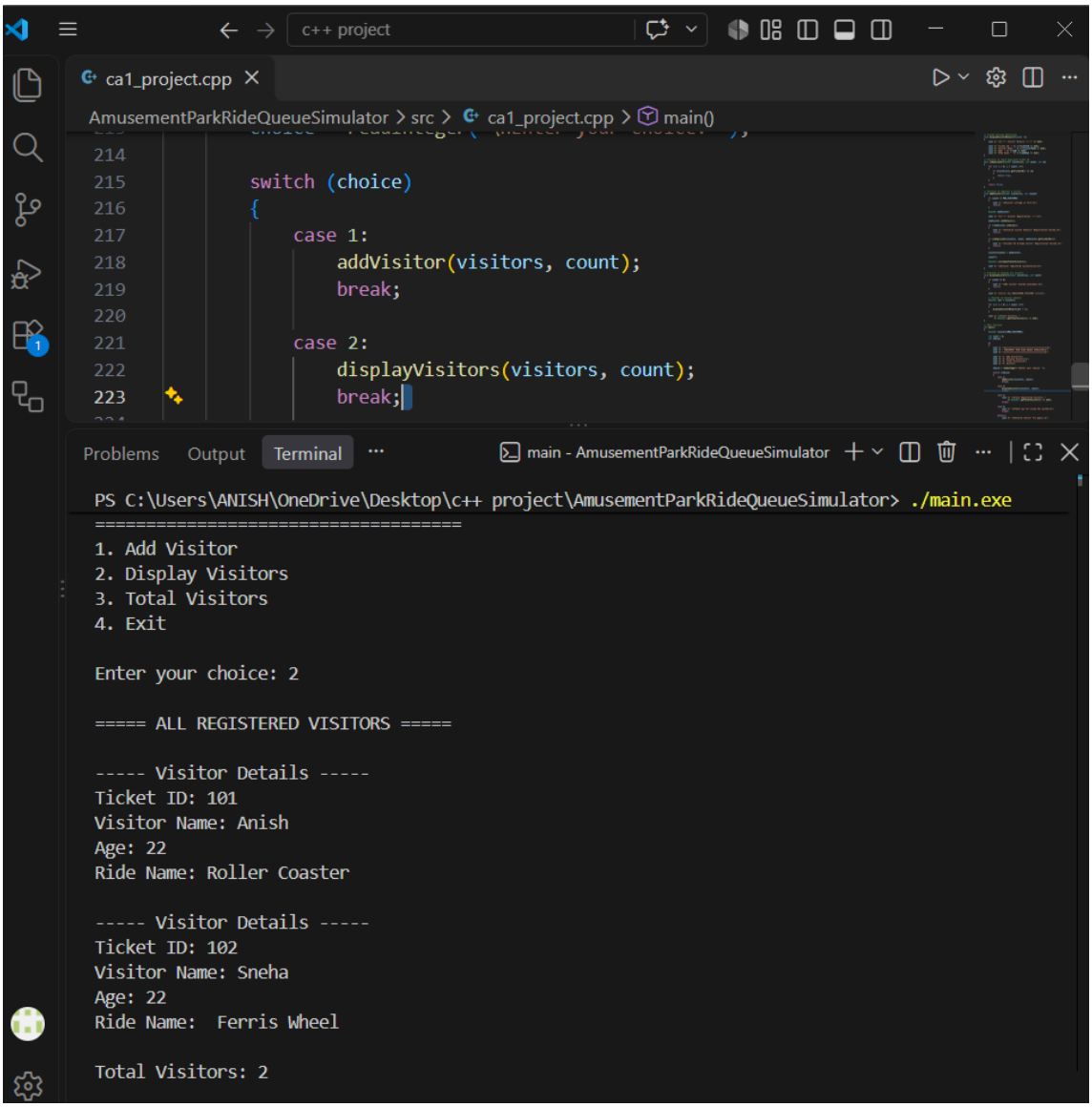
12.1 Main Menu Screenshot



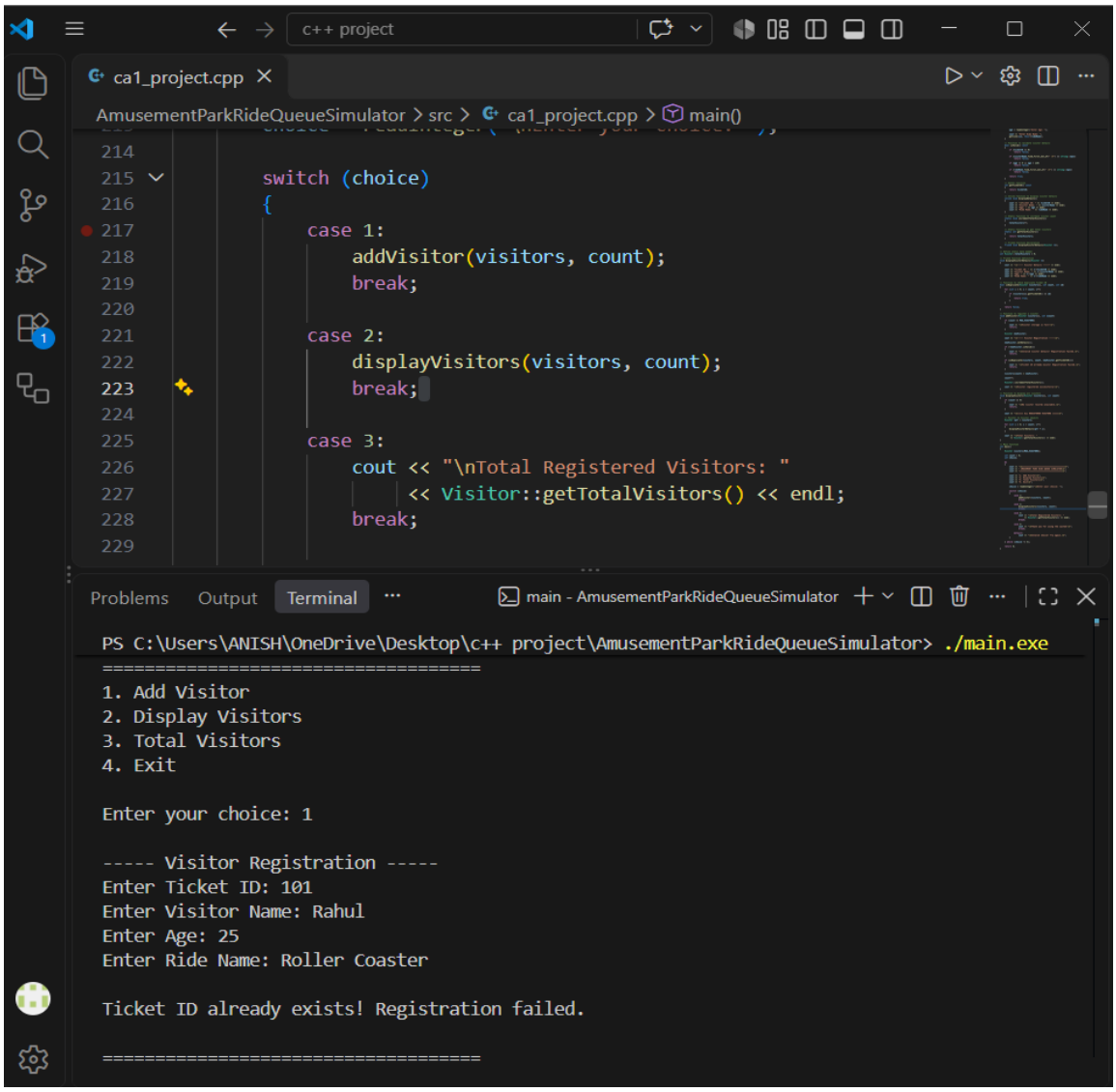
12.2 Successful Visitor Registration Screenshot



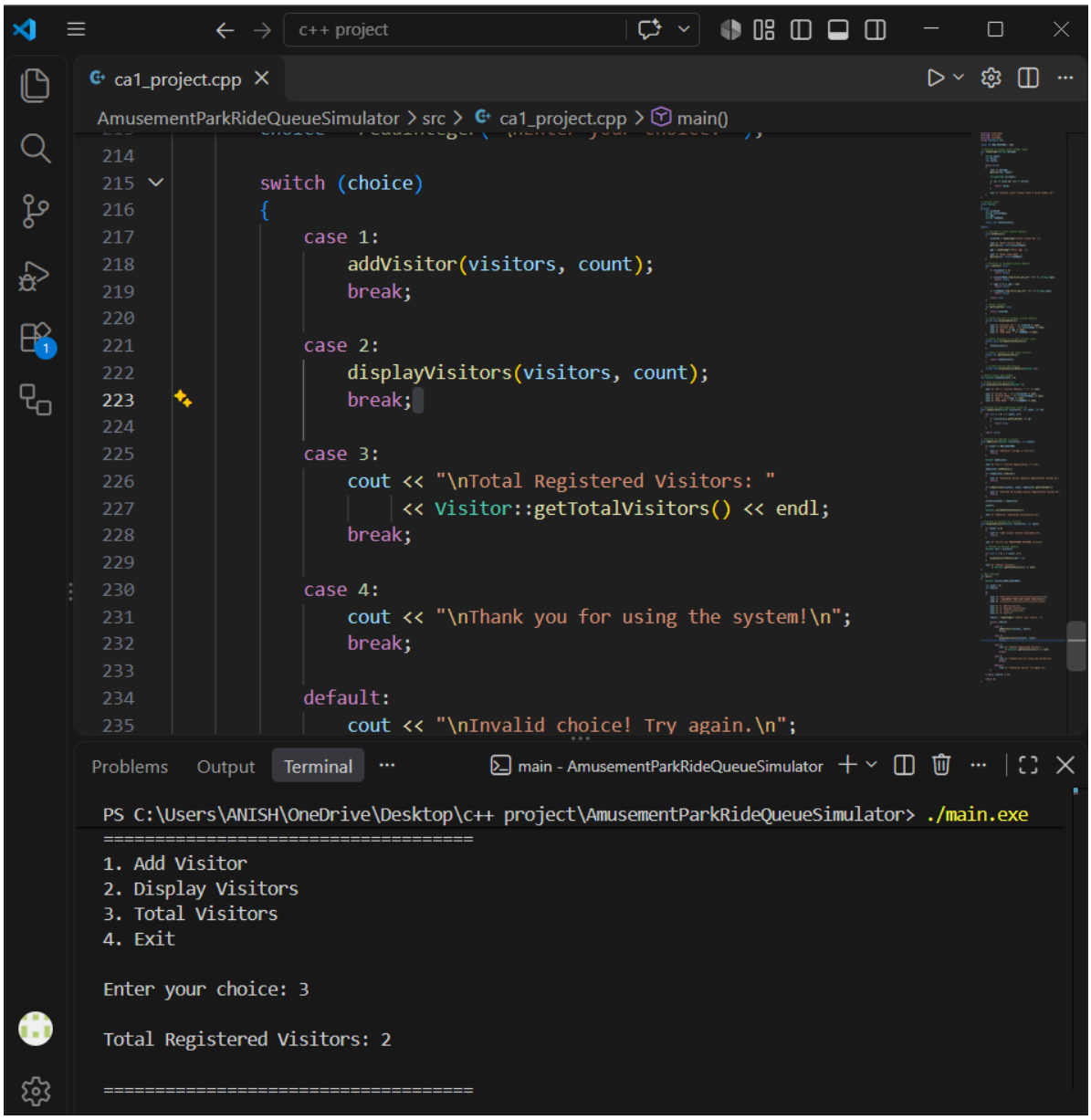
12.3 Display Visitor Records Screenshot



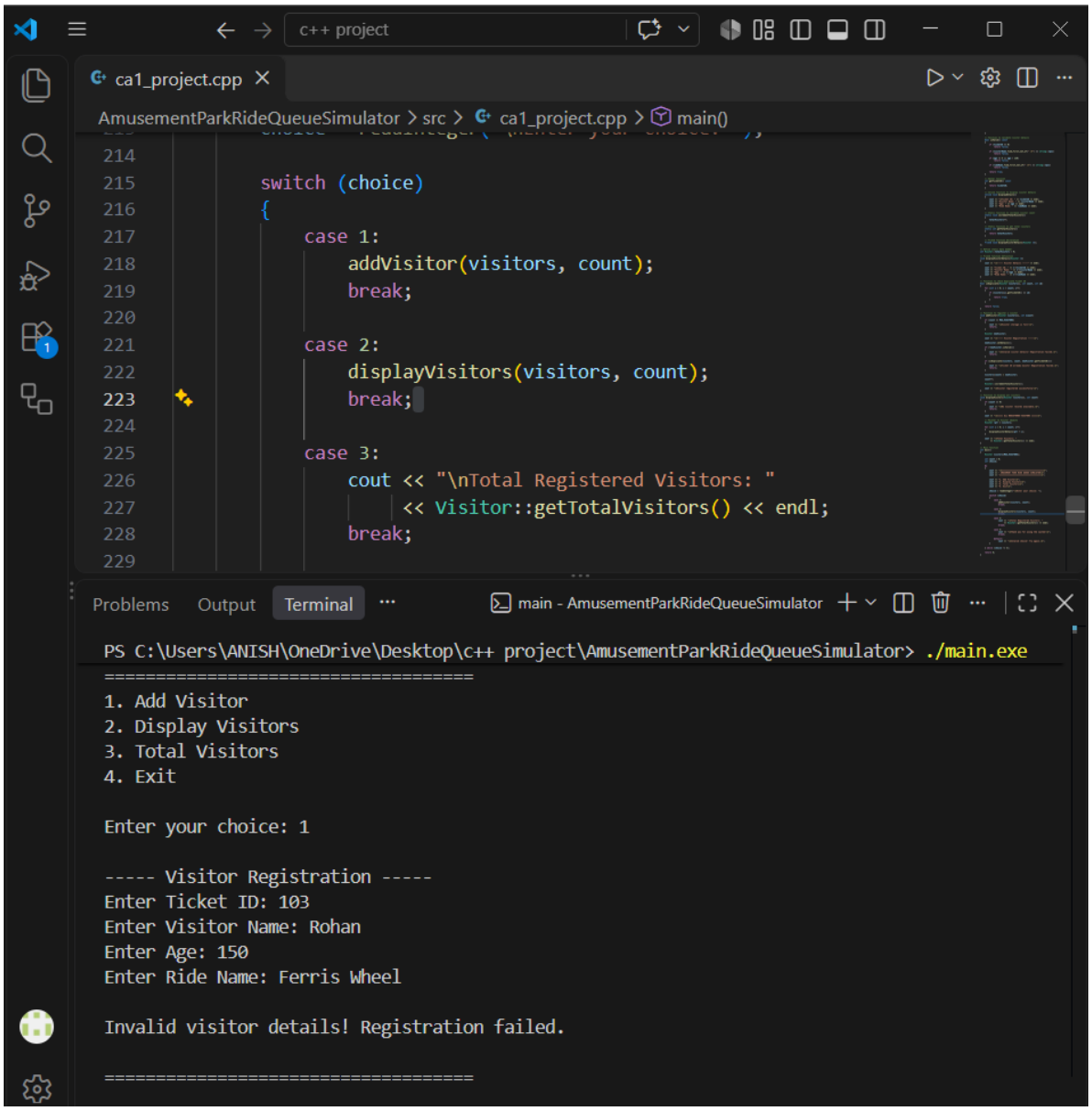
12.4 Duplicate Ticket ID Validation



12.5 Total Visitor Count



12.6 Invalid Input Validation



13. FUTURE ENHANCEMENTS

The current CA1 version provides the basic foundation of the Amusement Park Ride Queue Simulator. The following features can be added in future phases:

- File handling for permanent storage of visitor records.
- Update and delete operations for existing visitor records.
- Proper queue management for different rides.
- Ride capacity management and automatic ride allocation.

- Waiting-time calculation for visitors.
- Use of inheritance and polymorphism for different types of visitors or rides.
- Exception handling for better error management.
- Use of STL containers for more flexible data management.
- A graphical user interface for easier interaction.

14. CONCLUSION

The Amusement Park Ride Queue Simulator successfully provides a simple menu-driven system for registering visitors and managing their basic ride information. The CA1 implementation allows users to add visitor records, validate input, prevent duplicate ticket IDs, display registered visitors, and view the total number of visitors.

The project demonstrates the practical use of C++ and Object-Oriented Programming concepts such as classes, objects, encapsulation, arrays, functions, static members, pointers, and friend functions. Although the current version stores data only during program execution, it provides a strong foundation for adding advanced features in future phases.

Overall, the CA1 project achieves its main objectives and provides a functional base for further development of the Amusement Park Ride Queue Simulator.